

MODULE 3

Lossy Compression Algorithms - Distortion Measures, The Rate- Distortion Theory, Quantization, Transform Coding, Wavelet-Based Coding, Wavelet Packets.

Image Compression Standards— JPEG, JPEG2000, JPEG-LS, Bi-level Image Compression Standards.

Audio Compression Techniques— ADPCM in Speech Coding, Vocoder, Psychoacoustics, MPEG Audio.

In lossy compression, the compressed image is usually not the same as the original image but is meant to form a close approximation to the original image perceptually. To quantitatively describe how close the approximation is to the original data, some form of distortion measure is required.

Distortion Measures

- A distortion measure is a mathematical quantity that specifies how close an approximation is to its original, using some distortion criteria.
- When looking at compressed data, it is natural to think of the distortion in terms of the numerical difference between the original data and the reconstructed data.
- when the data to be compressed is an image, such a measure may not yield the intended result.
- For example, if the reconstructed image is the same as original image except that it is shifted to the right by one vertical scan line, an average human observer would have a hard time distinguishing it from the original and would therefore conclude that the distortion is small.

1. Mean Squared Error (MSE)

- MSE measures the average squared differences between the original and the compressed data.
- It is defined as :-

$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2$$

where x_n , y_n , and N are the input data sequence, reconstructed data sequence, and length of the data sequence, respectively.

2. signal-to-noise ratio (SNR)

- By taking the ratio of the average square of the original data sequence and the mean square error (MSE). In decibel units (dB), it is defined as

$$SNR = 10 \log_{10} \frac{\sigma_x^2}{\sigma^2}$$

where σ_x^2 is the average square value of the original data sequence

3. Peak-signal-to-noise ratio (PSNR)

- PSNR is a measure used to assess the quality of reconstruction. It is derived from MSE and is usually expressed in decibels (dB). which measures the size of the error relative to the peak value of the signal x_{peak} .

$$PSNR = 10 \log_{10} \frac{x_{\text{peak}}^2}{\sigma_d^2}.$$

The Rate-Distortion Theory

- **Rate (R):** The number of bits per symbol required to encode the source.
- **Distortion (D):** The error or loss incurred when reconstructing the signal after compression.
- Lossy compression always involves a tradeoff between rate and distortion. Higher compression (lower bit rate) leads to higher distortion (loss of quality). Lower distortion requires a higher bit rate. Within this framework, the tradeoff between rate and distortion is represented in the form of a rate-distortion function $R(D)$.

- **Rate-Distortion Function ($R(D)$):** The minimum bit rate required to achieve a given level of distortion. For a given source and a given distortion measure, if D is a tolerable amount of distortion, $R(D)$ specifies the lowest rate at which the source data can be encoded while keeping the distortion bounded above by D . It is easy to see that when $D = 0$, we have a lossless compression of the source.

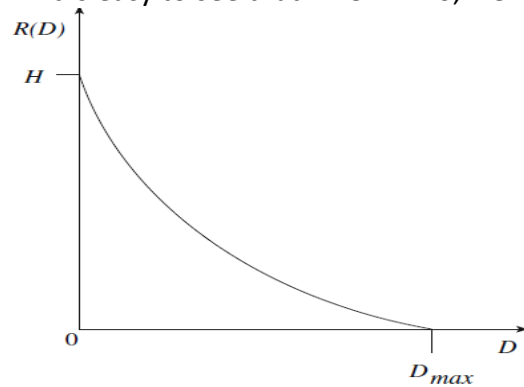


Fig. 8.1 Typical rate-distortion function

- Notice that the minimum possible rate at $D = 0$, no loss, is the entropy of the source data. The distortion corresponding to a rate $R(D) \Rightarrow 0$ is the maximum amount of distortion incurred when “nothing” is coded.

Quantization

- Quantization is the process of mapping a large set of input values to a smaller set, typically by rounding or truncating the values. This reduces the number of distinct values that need to be stored, which helps in compressing the data. Quantization in some form is the heart of any lossy scheme. Without quantization, we would indeed be losing little information.
- The input and output of each quantizer can be either scalar values or vector values, thus leading to **scalar quantizers and vector quantizers**

How it works:

- **Transform:** First, the data (e.g., an image or audio signal) is transformed into a domain that’s easier to compress (like the frequency domain using a Fourier transform, Discrete Cosine Transform (DCT), or Wavelet Transform).
- **Quantization:** After transformation, the coefficients (the transformed values) are quantized. Essentially, these coefficients are mapped to a smaller, discrete set of values. This step introduces approximation errors since the original continuous values are rounded off.
- **Encoding:** The quantized values are then encoded using entropy coding techniques like Huffman coding or arithmetic coding, which further compress the data.

➤ Uniform Scalar Quantization

- A uniform scalar quantizer partitions the domain of input values into equally spaced intervals, except possibly at the two outer intervals. The endpoints of partition intervals are called the quantizer’s decision boundaries. The output or reconstruction value corresponding to each interval is taken to be the midpoint of the interval. The length of each interval is referred to as the step size, denoted by the symbol θ .
- Uniform scalar quantizers are of two types: midrise and midtread.
- A midrise quantizer is used with an even number of output levels
- A midtread quantizer with an odd number. The midrise quantizer has a partition interval that brackets zero.
- The midtread quantizer has zero as one of its output values, hence, it is also known as dead-zone quantizer, because it turns a range of nonzero input values into the zero output.

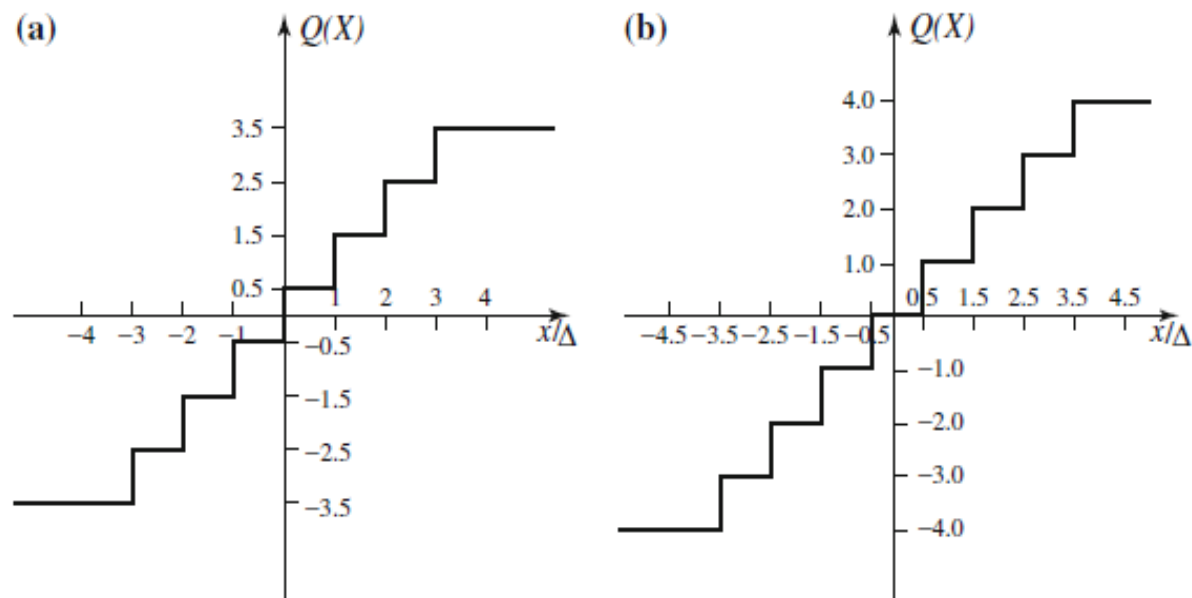


Fig. 8.2 Uniform scalar quantizers: **a** midrise; **b** midtread

A midtread quantizer is important when source data represents the zero value by fluctuating between small positive and negative numbers. Applying the midtread quantizer in this case would produce an accurate and steady representation of the value zero. For the special case $\Delta = 1$, we can simply compute the output values for these quantizers as

$$Q_{\text{midrise}}(x) = \lceil x \rceil - 0.5 \quad (8.4)$$

$$Q_{\text{midtread}}(x) = \lfloor x + 0.5 \rfloor \quad (8.5)$$

The goal for the design of a successful uniform quantizer is to minimize the distortion for a given source input with a desired number of output values. This can be done by adjusting the step size Δ to match the input statistics.

Let's examine the performance of an M level quantizer. Let $B = \{b_0, b_1, \dots, b_M\}$ be the set of decision boundaries and $Y = \{y_1, y_2, \dots, y_M\}$ be the set of reconstruction or output values. Suppose the input is uniformly distributed in the interval $[-X_{\max}, X_{\max}]$. The rate of the quantizer is

$$R = \lceil \log_2 M \rceil \quad (8.6)$$

That is, R is the number of bits required to code M things — in this case, the M output levels.

The step size Δ is given by

$$\Delta = \frac{2X_{\max}}{M} \quad (8.7)$$

since the entire range of input values is from $-X_{\max}$ to $X_{\max}$. For bounded input, the quantization error caused by the quantizer is referred to as *granular distortion*. If the quantizer replaces a whole range of values, from a maximum value to ∞ , and similarly for negative values, that part of the distortion is called the *overload* distortion.

Consider a **3-bit uniform quantizer** with levels: $-3, -2, -1, 0, 1, 2, 3$

If an input signal value is **0.6**, it gets rounded to **1**, introducing a **small error (0.4)**. These small errors in the quantization process create granular distortion.

Assume a **3-bit quantizer** with a range from **-3 to 3**. If an input signal has a value of **4.2**, it is outside the range. The quantizer will **clip** it to **3**, causing a **large error (1.2)**.

➤ Nonuniform Scalar Quantization

- If the input source is not uniformly distributed, a uniform quantizer may be inefficient. Increasing the number of decision levels within the region where the source is densely distributed can effectively lower granular distortion. In addition, without having to increase the total number of decision levels, we can enlarge the region in which the source is sparsely distributed. Such nonuniform quantizers thus have nonuniformly defined decision boundaries.

There are two common approaches for nonuniform quantization:

- Lloyd–Max quantizer
- Companded quantizer

Lloyd–Max Quantizer

If the input source is not uniformly distributed, a uniform quantizer may be inefficient. Increasing the number of decision levels within the region where the source is densely distributed can effectively lower granular distortion. In addition, without having to increase the total number of decision levels, we can enlarge the region in which the source is sparsely distributed. Such *nonuniform quantizers* thus have nonuniformly defined decision boundaries.

Lloyd–Max Quantizer.* For a uniform quantizer, the total distortion is equal to the granular distortion, as in Eq. (8.8). If the source distribution is not uniform, we must explicitly consider its probability distribution (*probability density function*) $f_X(x)$. Now we need the correct decision boundaries b_i and reconstruction values y_i , by solving for both simultaneously. To do so, we plug variables b_i , y_i into a total distortion measure

$$D_{gran} = \sum_{j=1}^M \int_{b_{j-1}}^{b_j} (x - y_j)^2 \frac{1}{X_{max}} f_X(x) dx \quad (8.12)$$

Then we can minimize the total distortion by setting the derivative of Eq. (8.12) to zero. Differentiating with respect to y_j yields the set of reconstruction values

$$y_j = \frac{\int_{b_{j-1}}^{b_j} x f_X(x) dx}{\int_{b_{j-1}}^{b_j} f_X(x) dx} \quad (8.13)$$

This says that the optimal reconstruction value is the weighted centroid of the x interval. Differentiating with respect to b_j and setting the result to zero yields

$$b_j = \frac{y_{j+1} + y_j}{2} \quad (8.14)$$

This gives a decision boundary b_j at the midpoint of two adjacent reconstruction values. Solving these two equations simultaneously is carried out by iteration. The result is termed the Lloyd–Max quantizer.

The **Lloyd-Max quantizer** is an optimal **non-uniform quantizer** designed to minimize the mean squared quantization error. It is an improvement over **uniform quantization**, particularly for signals with **non-uniform probability distributions** (e.g., speech, images).

ALGORITHM 8.1 LLOYD-MAX QUANTIZATION

```

BEGIN
  Choose initial level set  $y_0$ 
   $i = 0$ 
  Repeat
    Compute  $b_i$  using Equation 8.14
     $i = i + 1$ 
    Compute  $y_i$  using Equation 8.13
  Until  $|y_i - y_{i-1}| < \epsilon$ 
END

```

Let's apply the **Lloyd-Max quantization algorithm** to the given input:

$X = \{1, 2, 3, 8, 9, 10\}$

We will use **2 quantization levels** as an example.

Step 1: Initialize Quantization Levels

A reasonable **initial guess** for quantization levels is:

$Y = \{2, 9\}$

since the data seems to have two clusters: **(1,2,3)** and **(8,9,10)**.

Step 2: Compute Decision Boundaries

Boundaries are computed as the midpoint between quantization levels:

$t_1 = 2 + 9 / 2 = 5.5$

So, the **decision boundary** is **5.5**, dividing the input into two groups:

- Values $\leq 5.5 \rightarrow$ assigned to 2
- Values $> 5.5 \rightarrow$ assigned to 9

Step 3: Assign Values to Nearest Level

Now, we assign each value in X to the closest quantization level:

Input X	Boundary $t = 5.5$	Assigned Level
1	≤ 5.5	2
2	≤ 5.5	2
3	≤ 5.5	2
8	> 5.5	9
9	> 5.5	9
10	> 5.5	9

Step 4: Compute New Reconstruction Levels (Centroids)

We compute the new quantization levels by taking the mean of values assigned to each group:

$$Y_1 = \frac{(1 + 2 + 3)}{3} = \frac{6}{3} = 2$$
$$Y_2 = \frac{(8 + 9 + 10)}{3} = \frac{27}{3} = 9$$

So, the updated quantization levels remain the same:

$$Y = \{2, 9\}$$

Since there is **no change**, the algorithm **converges**, and we stop here.

Final Quantized Values

Original X	Quantized $\hat{X}$
1	2
2	2
3	2
8	9
9	9
10	9

The final quantized values are {2, 2, 2, 9, 9, 9}.

Companded Quantizer

Companded Quantizer. In companded quantization, the input is mapped by a *compressor function* G and then quantized using a uniform quantizer. After transmission, the quantized values are mapped back using an *expander function* G^{-1} . The block diagram for the companding process is shown in Figure 8.4, where $\hat{X}$ is the quantized version of X . If the input source is bounded by x_{max} , then any nonuniform quantizer can be represented as a companded quantizer. The two commonly used companders are the μ -law and A-law companders (Section 6.1).

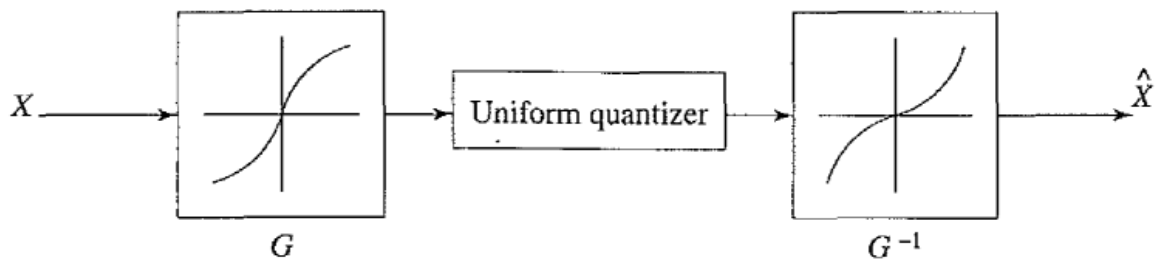


FIGURE 8.4: Companded quantization.

Vector Quantization*

One of the fundamental ideas in Shannon's original work on information theory is that any compression system performs better if it operates on vectors or groups of samples rather than on individual symbols or samples. We can form vectors of input samples by concatenating a number of consecutive samples into a single vector. For example, an input vector might be a segment of a speech sample, a group of consecutive pixels in an image, or a chunk of data in any other format.

The idea behind *vector quantization* (VQ) is similar to that of scalar quantization but extended into multiple dimensions. Instead of representing values within an interval in one-dimensional space by a reconstruction value, as in scalar quantization, in VQ an n -component *code vector* represents vectors that lie within a region in n -dimensional space. A collection of these code vectors forms the *codebook* for the vector quantizer.

Since there is no implicit ordering of code vectors, as there is in the one-dimensional case, an index set is also needed to index into the codebook. Figure 8.5 shows the basic vector quantization procedure. In the diagram, the encoder finds the closest code vector to the input vector and outputs the associated index. On the decoder side, exactly the same codebook is used. When the coded index of the input vector is received, a simple table lookup is performed to determine the reconstruction vector.

Finding the appropriate codebook and searching for the closest code vector at the encoder end may require considerable computational resources. However, the decoder can execute quickly, since only a constant time operation is needed to obtain the reconstruction. Because of this property, VQ is attractive for systems with a lot of resources at the encoder end while

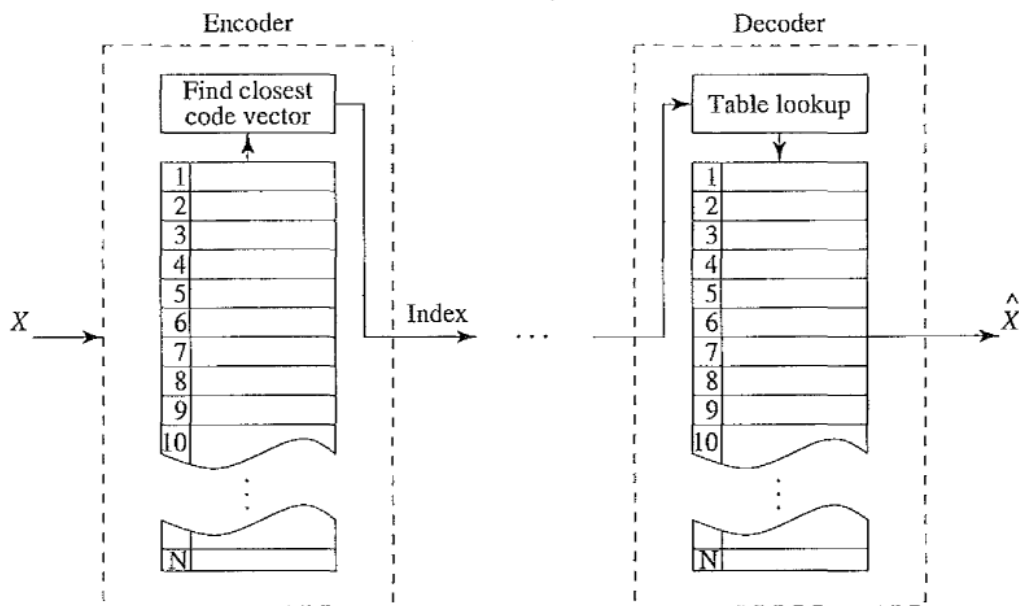


FIGURE 8.5: Basic vector quantization procedure.

Transform Coding

From basic principles of information theory, we know that coding vectors is more efficient than coding scalars (see Section 7.4.2). To carry out such an intention, we need to group blocks of consecutive samples from the source input into vectors.

Let $\mathbf{X} = \{x_1, x_2, \dots, x_k\}^T$ be a vector of samples. Whether our input data is an image, a piece of music, an audio or video clip, or even a piece of text, there is a good chance that a substantial amount of correlation is inherent among neighboring samples x_i . The rationale behind transform coding is that if $\mathbf{Y}$ is the result of a linear transform $\mathbf{T}$ of the input vector $\mathbf{X}$ in such a way that the components of $\mathbf{Y}$ are much less correlated, then $\mathbf{Y}$ can be coded more efficiently than $\mathbf{X}$.

Generally, the transform $\mathbf{T}$ itself does not compress any data. The compression comes from the processing and *quantization* of the components of $\mathbf{Y}$. In this section, we will study the Discrete Cosine Transform (DCT) as a tool to decorrelate the input signal. We will also examine the Karhunen–Loève Transform (KLT), which *optimally* decorrelates the components of the input $\mathbf{X}$.

➤ DCT

Discrete Cosine Transform (DCT)

The Discrete Cosine Transform (DCT), a widely used transform coding technique, is able to perform decorrelation of the input signal in a data-independent manner. Because of this, it has gained tremendous popularity. We will examine the definition of the DCT and discuss some of its properties, in particular the relationship between it and the more familiar Discrete Fourier Transform (DFT).

Definition of DCT. Let's start with the two-dimensional DCT. Given a function $f(i, j)$ over two integer variables i and j (a piece of an image), the 2D DCT transforms it into a new function $F(u, v)$, with integer u and v running over the same range as i and j . The general definition of the transform is

$$F(u, v) = \frac{2 C(u) C(v)}{\sqrt{MN}} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \cos \frac{(2i+1)u\pi}{2M} \cos \frac{(2j+1)v\pi}{2N} f(i, j) \quad (8.15)$$

where $i, u = 0, 1, \dots, M-1$, $j, v = 0, 1, \dots, N-1$, and the constants $C(u)$ and

$$C(\xi) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } \xi = 0, \\ 1 & \text{otherwise.} \end{cases} \quad (8.16)$$

$$F(u, v) = \frac{C(u) C(v)}{4} \sum_{i=0}^7 \sum_{j=0}^7 \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} f(i, j), \quad (8.17)$$

where $i, j, u, v = 0, 1, \dots, 7$, and the constants $C(u)$ and $C(v)$ are determined by Eq. (8.16).

2D Inverse Discrete Cosine Transform (2D IDCT)

The inverse function is almost the same, with the roles of $f(i, j)$ and $F(u, v)$ reversed, except that now $C(u)C(v)$ must stand inside the sums:

$$\tilde{f}(i, j) = \sum_{u=0}^7 \sum_{v=0}^7 \frac{C(u)C(v)}{4} \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} F(u, v) \quad (8.18)$$

where $i, j, u, v = 0, 1, \dots, 7$, and the constants $C(u)$ and $C(v)$ are determined by Eq. (8.16).

The 2D transforms are applicable to 2D signals, such as digital images. As shown below, the 1D version of the DCT and IDCT is similar to the 2D version.

1D Discrete Cosine Transform (1D DCT)

$$F(u) = \frac{C(u)}{2} \sum_{i=0}^7 \cos \frac{(2i+1)u\pi}{16} f(i), \quad (8.19)$$

where $i = 0, 1, \dots, 7$, $u = 0, 1, \dots, 7$, and the constant $C(u)$ is the same as in Eq. (8.16).

1D Inverse Discrete Cosine Transform (1D-IDCT)

$$\tilde{f}(i) = \sum_{u=0}^7 \frac{C(u)}{2} \cos \frac{(2i+1)u\pi}{16} F(u), \quad (8.20)$$

➤ Karhunen–Loève Transform*

- The Karhunen–Loève Transform (KLT), also known as the Hotelling Transform, is a mathematical technique used to transform a set of random variables into a new set of uncorrelated variables, while preserving the variance (or energy) of the original data. In the KLT, the data is transformed using the eigenvectors of the covariance matrix of the data, which represent the directions of maximum variance. This results in a transformation where the new variables (or components) are uncorrelated and ordered by the amount of variance they explain.
- The Karhunen–Loève Transform (KLT) is a reversible linear transform that exploits the statistical properties of the vector representation. Its primary property is that it optimally decorrelates the input.
- To do so, it fits an n-dimensional ellipsoid around the (mean-subtracted) data.
- The main ellipsoid axis is the major direction of change in the data.
- To understand the optimality of the KLT, consider the autocorrelation matrix R_X of the set of k input vectors X , defined in terms of the expectation value $E(\cdot)$ as

$$R_X = E[XX^T] \quad (8.36)$$

$$= \begin{bmatrix} R_X(1, 1) & R_X(1, 2) & \cdots & R_X(1, k) \\ R_X(2, 1) & R_X(2, 2) & \cdots & R_X(2, k) \\ \vdots & \vdots & \ddots & \vdots \\ R_X(k, 1) & R_X(k, 2) & \cdots & R_X(k, k) \end{bmatrix} \quad (8.37)$$

where $R_X(t, s) = E[X_t X_s]$ is the autocorrelation function. Our goal is to find a transform $\mathbf{T}$ such that the components of the output $\mathbf{Y}$ are uncorrelated—that is, $E[Y_t Y_s] = 0$, if $t \neq s$. Thus, the autocorrelation matrix of $\mathbf{Y}$ takes on the form of a positive diagonal matrix.

Since any autocorrelation matrix is symmetric and nonnegative definite, there are k orthogonal eigenvectors $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k$ and k corresponding real and nonnegative eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k \geq 0$. We define the Karhunen-Loève transform as

$$\mathbf{T} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k]^T. \quad (8.38)$$

Then, the autocorrelation matrix of $\mathbf{Y}$ becomes

$$\mathbf{R}_Y = E[\mathbf{Y}\mathbf{Y}^T] \quad (8.39)$$

$$= E[\mathbf{T}\mathbf{X}\mathbf{X}^T\mathbf{T}] \quad (8.40)$$

$$= \mathbf{T}\mathbf{R}_X\mathbf{T}^T \quad (8.41)$$

$$= \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ 0 & \vdots & \ddots & 0 \\ 0 & 0 & \dots & \lambda_k \end{bmatrix}. \quad (8.42)$$

WAVELET BASED CODING

- Decomposing the input signal into its constituents allows us to apply coding techniques suitable for each constituent, to improve compression performance. Consider a time-dependent signal $f(t)$. The traditional method of signal decomposition is the Fourier transform.
- Another method of decomposition that has gained a great deal of popularity in recent years is the wavelet transform. It seeks to represent a signal with good resolution in both time and frequency, by using a set of basis functions called wavelets.
- There are two types of wavelet transforms: the **Continuous Wavelet Transform (CWT)** and the **Discrete Wavelet Transform (DWT)**.
- CWT is applied to the large class of functions $f(x)$ that are square integrable on the real line.

$$\int [f(x)]^2 dx < \infty.$$

- In mathematics, this is written as

$$f(x) \in L^2(R)$$

Simple Wavelet Transform

- The objective of the wavelet transform is to decompose the input signal, for compression purposes, into components that are easier to deal with, have special interpretations, or have some components that can be thresholded away.
- Suppose we are given the following input sequence:

$$\{x_{n,i}\} = \{10, 13, 25, 26, 29, 21, 7, 15\} \quad (8.39)$$

Here, $i \in [0..7]$ indexes “pixels”, and n stands for the level of a *pyramid* we are on. At the top, $n = 3$ for this sequence, and we shall form three more sequences, for $n = 2, 1$, and 0 . At each level, less information will be retained in the beginning elements of the transformed signal sequence. When we reach pyramid level $n = 0$, we end up with the sequence average stored in the first element. The remaining elements store detail information.

Consider the transform that replaces the original sequence with its pairwise *average* $x_{n-1,i}$ and *difference* $d_{n-1,i}$, defined as follows:

$$x_{n-1,i} = \frac{x_{n,2i} + x_{n,2i+1}}{2} \quad (8.40)$$

$$d_{n-1,i} = \frac{x_{n,2i} - x_{n,2i+1}}{2} \quad (8.41)$$

Notice that the averages and differences are applied only on consecutive *pairs* of input sequences whose first element has an even index. Therefore, the number of elements in each set $\{x_{n-1,i}\}$ and $\{d_{n-1,i}\}$ is exactly half the number of elements in the original sequence. We can form a new sequence having length equal to that of the original sequence by concatenating the two sequences $\{x_{n-1,i}\}$ and $\{d_{n-1,i}\}$. The resulting sequence is thus

$$\{x_{n-1,i}, d_{n-1,i}\} = \{11.5, 25.5, 25, 11, -1.5, -0.5, 4, -4\} \quad (8.42)$$

where we are now at level $n - 1 = 2$. This sequence has exactly the same number of elements as the input sequence — the transform did not increase the amount of data. Since the first half of the above sequence contains averages from the original sequence, we can view it as a coarser approximation to the original signal.

The second half of this sequence can be viewed as the details or approximation errors of the first half. Most of the values in the detail sequence are much smaller than those of the original sequence. Thus, most of the energy is effectively concentrated in the first half. Therefore, we can potentially store $\{d_{n-1,i}\}$ using fewer bits.

It is easily verified that the original sequence can be reconstructed from the transformed sequence, using the relations

$$\begin{aligned} x_{n,2i} &= x_{n-1,i} + d_{n-1,i} \\ x_{n,2i+1} &= x_{n-1,i} - d_{n-1,i} \end{aligned} \quad (8.43)$$

This transform is the discrete Haar wavelet transform. Averaging and differencing can be carried out by applying a so-called *scaling function* and *wavelet function* along the signal. Figure 8.12 shows the Haar version of these functions.

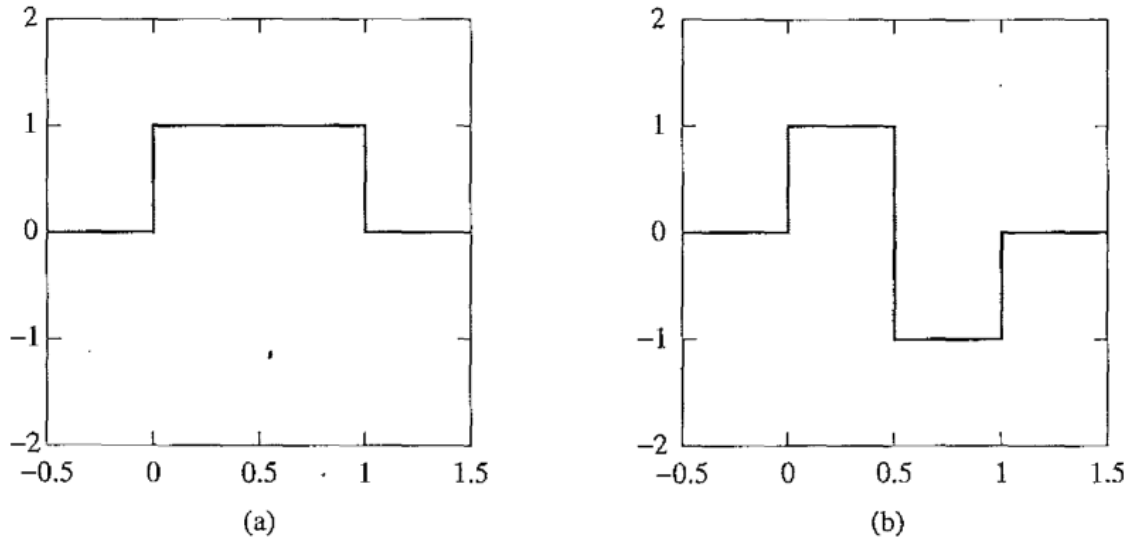


FIGURE 8.12: Haar Wavelet Transform: (a) scaling function; (b) wavelet function.

We can further apply the same transform to $\{x_{n-1,i}\}$, to obtain another level of approximation $x_{n-2,i}$ and detail $d_{n-2,i}$:

$$\{x_{n-2,i}, d_{n-2,i}, d_{n-1,i}\} = \{18.5, 18, -7, 7, -1.5, -0.5, 4, -4\} \quad (8.44)$$

This is the essential idea of multiresolution analysis. We can now study the input signal in three different scales, along with the details needed to go from one scale to another. This process can continue n times, until only one element is left in the approximation sequence. In this case, $n = 3$, and the final sequence is given below:

$$\{x_{n-3,i}, d_{n-3,i}, d_{n-2,i}, d_{n-1,i}\} = \{18.25, 0.25, -7, 7, -1.5, -0.5, 4, -4\} \quad (8.45)$$

Now we realize that n was 3 because only three resolution changes were available until we reached the final form.

Continuous Wavelet Transform*

We noted that the motivation for the use of wavelets is to provide a set of basis functions that decompose a signal in time over parameters in the frequency domain and the time domain simultaneously. A Fourier transform aims to pin down only the frequency content of a signal, in terms of spatially varying rather than time varying signals. What wavelets aim to do is pin down the frequency content at different parts of the image.

For example, one part of the image may be “busy” with texture and thus high-frequency content, while another part may be smooth, with little high-frequency content. Naturally, one can think of obvious ways to consider frequencies for localized areas of an image: divide an image into parts and fire away with Fourier analysis. The time-sequence version of that idea is called the *Short-Term* (or *Windowed*) *Fourier Transform*. And other ideas have also arisen. However, it turns out that wavelets, a much newer development, have neater characteristics.

In general, a wavelet is a function $\psi \in L^2(R)$ with a zero average,

$$\int_{-\infty}^{+\infty} \psi(t) dt = 0 \quad (8.49)$$

The function ψ is normalized with $\|\psi\| = 1$ and centered in the neighborhood of $t = 0$. We can obtain a *family* of wavelet functions by scaling and translating the *mother wavelet* ψ as follows:

$$\psi_{s,u}(t) = \frac{1}{\sqrt{s}} \psi\left(\frac{t-u}{s}\right) \quad (8.51)$$

If $\psi(t)$ is normalized, so is $\psi_{s,u}(t)$.

The *Continuous Wavelet Transform* (CWT) of $f \in L^2(R)$ at time u and scale s is defined as

$$\mathcal{W}(f, s, u) = \int_{-\infty}^{+\infty} f(t) \psi_{s,u}(t) dt \quad (8.52)$$

The CWT of a 1D signal is a 2D function — a function of both *scale* s and *shift* u .

Discrete Wavelet Transform*

- Discrete wavelets are again formed from a mother wavelet, but with scale and shift in discrete steps. Multiresolution Analysis and the Discrete Wavelet Transform
- The connection between wavelets in the continuous time domain and filter banks in the discrete time domain is multiresolution analysis.
- It is possible to construct wavelets ψ such that the dilated and translated family.

$$\left\{ \psi_{j,n}(t) = \frac{1}{\sqrt{2^j}} \psi\left(\frac{t - 2^j n}{2^j}\right) \right\}_{(j,n) \in \mathbb{Z}^2}$$

Wavelet Packets

Wavelet packets can be viewed as a generalization of wavelets. They were first introduced by Coifman, Meyer, Quake, and Wickerhauser [9] as a family of orthonormal bases for discrete functions of $\mathbb{R}^N$. A complete subband decomposition can be viewed as a decomposition of the input signal, using an analysis tree of depth $\log N$.

In the usual dyadic wavelet decomposition, only the low-pass-filtered subband is recursively decomposed and thus can be represented by a logarithmic tree structure. However, a wavelet packet decomposition allows the decomposition to be represented by any pruned subtree of the full tree topology. Therefore, this representation of the decomposition topology is isomorphic to all permissible subband topologies [10]. The leaf nodes of each pruned subtree represent one permissible orthonormal basis.

The wavelet packet decomposition offers a number of attractive properties, including

- Flexibility, since a best wavelet basis in the sense of some cost metric can be found within a large library of permissible bases
- Favorable localization of wavelet packets in both frequency and space.
- Low computational requirement for wavelet packet decomposition, because each decomposition can be computed in the order of $N \log N$ using fast filter banks.

Wavelet packets are currently being applied to solve various practical problems such as image compression, signal de-noising, fingerprint identification, and so on.

IMAGE COMPRESSION STANDARDS

The JPEG Standard

- JPEG is an image compression standard developed by the Joint Photographic Experts Group. It was formally accepted as an international standard in 1992.

Main Steps in JPEG Image Compression

- Transform RGB to YCbCr and subsample color
- Perform DCT on image blocks
- Apply Quantization
- Perform Zigzag ordering and run-length encoding
- Perform Entropy coding

1. Color Space Conversion

- JPEG images are usually represented in the RGB color space, but for better compression, the image is often converted to YCbCr color space.
- Y represents the luminance (brightness), and Cb and Cr represent chrominance (color information).
- This conversion is done because the human eye is more sensitive to brightness than color, allowing for more aggressive compression of the chrominance channels.

2. Downsampling (Subsampling)

- Often, the chrominance components (Cb and Cr) are downsampled to reduce their resolution. Commonly, a 4:2:0 or 4:2:2 subsampling method is used.
- This reduces the amount of chrominance data while maintaining perceived image quality, taking advantage of the human eye's less sensitivity to color differences.

3. Block Splitting

- The image is divided into 8x8 pixel blocks for processing. Each block is treated independently, which helps in compressing the image efficiently by applying similar operations to each block.

4. Discrete Cosine Transform (DCT)

- The Discrete Cosine Transform is applied to each 8x8 block of pixels. This step transforms the image data from the spatial domain (pixels) into the frequency domain.
- The DCT helps separate the image into parts of varying importance based on frequency. Low-frequency components (like edges and smooth areas) are preserved, while high-frequency components (like noise or fine details) can be discarded or compressed more heavily.

5. Quantization

- After the DCT, the frequency coefficients are quantized. This is a lossy compression step where less important frequencies (those with high frequency) are reduced in precision or discarded.
- A quantization matrix is applied, which defines how much each frequency component should be compressed. Typically, higher frequencies are quantized more heavily, leading to more compression.
- The result is that the quantized values for each block will have fewer bits, which helps to reduce the image size.

6. Zigzag Scan and Run-Length Encoding

- After quantization, the 8x8 block is scanned in a zigzag pattern to convert the 2D matrix of values into a 1D array of coefficients.
- The 1D array is then encoded using run-length encoding (RLE), which groups consecutive zeros together, further reducing the size.

7. Huffman Coding

- The run-length encoded values are then encoded using Huffman coding, a lossless technique that assigns shorter codes to more frequent values and longer codes to less frequent ones.
- This step ensures that the most common patterns in the image are represented with the fewest bits, further compressing the image.

8. File Formatting

- The compressed image data is stored in the JPEG file format, along with necessary headers and metadata (such as quantization tables, Huffman tables, and other information about how the image was compressed).

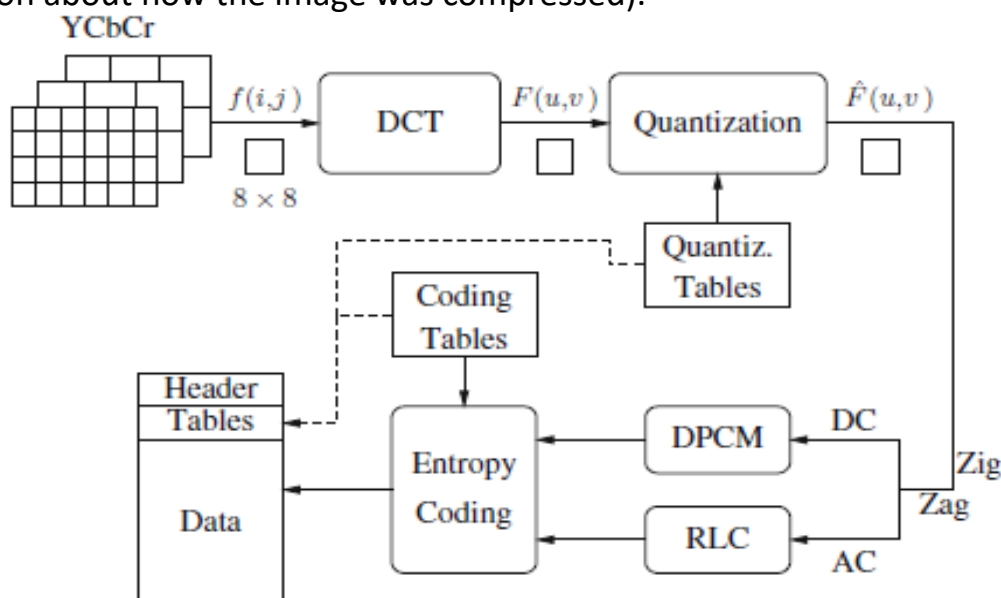


Fig.9.1 Block diagram for JPEG encoder

➤ DCT on Image Blocks

- Each image is divided into 8×8 blocks. The 2D DCT is applied to each block image $f(i, j)$, with output being the DCT coefficients $F(u, v)$ for each block.
- The choice of a small block size in JPEG is a compromise reached by the committee: a number larger than 8 would have made accuracy at low frequencies better, but using 8 makes the DCT (and IDCT) computation very fast.
- Using blocks at all, however, has the effect of isolating each block from its neighboring context. This is why JPEG images look choppy (“blocky”) when the user specifies a high compression ratio

➤ QUANTIZATION

Quantization. The quantization step in JPEG is aimed at reducing the total number of bits needed for a compressed image [2]. It consists of simply dividing each entry in the frequency space block by an integer, then rounding:

$$\hat{F}(u, v) = \text{round} \left(\frac{F(u, v)}{Q(u, v)} \right) \quad (9.1)$$

Here, $F(u, v)$ represents a DCT coefficient, $Q(u, v)$ is a quantization matrix entry, and $\hat{F}(u, v)$ represents the quantized DCT coefficients JPEG will use in the succeeding entropy coding.

➤ Preparation for Entropy Encoding

In Discrete Cosine Transform (DCT), the DC and AC coefficients represent different frequency components of an image block.

- ❑ The DC coefficient is the first value (top-left corner) of the DCT matrix.
- ❑ It represents the average brightness (intensity) of the entire 8×8 block.
- ❑ The remaining coefficients in the DCT matrix (except the DC value) are AC coefficients.
- ❑ They represent variations, edges, and details in the image. these coefficients can be quantized (many values become zero), leading to compression.

DC components are encoded separately using DPCM.

- AC components are encoded using Run-Length Encoding (RLE) and Huffman coding to remove redundancies.

Zigzag Ordering

The zigzag scan reorders the 8×8 matrix into a 1D array, prioritizing low-frequency components first. Zigzag Ordering is used to rearrange the DCT coefficients into a 1D sequence to optimize Run-Length Encoding (RLE).

Why Zigzag Ordering?

- Groups low-frequency coefficients first (important for image quality).
- High-frequency coefficients (mostly zeros) are placed at the end.
- Helps Run-Length Encoding (RLE) efficiently compress zero values.
- Eg: [100,50,30,20,0,0,10,0,0,0,0,5,0,0,0,0,0,0,0,0,0,0,0]

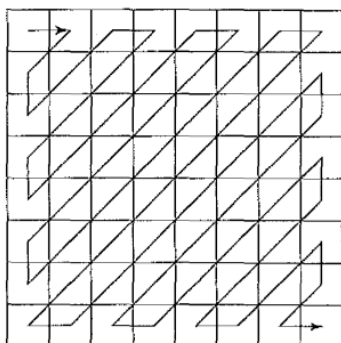


FIGURE 9.4: Zigzag scan in JPEG.

The RLC step replaces values by a pair (RUNLENGTH, VALUE) for each run of zeros in the AC coefficients of $\hat{F}$, where RUNLENGTH is the number of zeros in the run and VALUE is the next nonzero coefficient. To further save bits, a special pair (0,0) indicates the end-of-block after the last nonzero AC coefficient is reached. In the above example, not considering the first (DC) component, we will thus have

In the above example:

DC component: 100

☐ AC components in (zero count,value) format:

- (0, 50): 50 (no preceding zeros)
- (0, 30): 30 (no preceding zeros)
- (0,20): 20 (no preceding zeros)
- (2,10): 10 appears after two zeros
- (4,5): 5 appears after four zeros
- (EOB): Remaining values are zeros, so we stop.

➤ DPCM

Differential Pulse Code Modulation (DPCM) on DC Coefficients. The DC coefficients are coded separately from the AC ones. Each 8×8 image block has only one DC coefficient. The values of the DC coefficients for various blocks could be large and different, because the DC value reflects the average intensity of each block, but consistent with Observation 1 above, the DC coefficient is unlikely to change drastically within a short distance. This makes DPCM an ideal scheme for coding the DC coefficients.

If the DC coefficients for the first five image blocks are 150, 155, 149, 152, 144, DPCM would produce 150, 5, -6, 3, -8, assuming the predictor for the i th block is simply $d_i = DC_{i+1} - DC_i$, and $d_0 = DC_0$. We expect DPCM codes to generally have smaller magnitude and variance, which is beneficial for the next *entropy coding* step.

Huffman Coding of DC Coefficients Each DPCM-coded DC coefficient is represented by a pair of symbols (SIZE, AMPLITUDE), where SIZE indicates how many bits are needed for representing the coefficient and AMPLITUDE contains the actual bits.

DPCM values could require more than 8 bits and could be negative values. The one's-complement scheme is used for negative numbers — that is, binary code 10 for 2, 01 for -2; 11 for 3, 00 for -3; and so on. In the example we are using, codes 150, 5, -6, 3, -8 will be turned into

(8, 10010110), (3, 101), (3, 001), (2, 11), (4, 0111)

Huffman Coding of AC Coefficients. Recall we said that the AC coefficients are run-length coded and are represented by pairs of numbers (RUNLENGTH, VALUE). However, in an actual JPEG implementation, VALUE is further represented by SIZE and AMPLITUDE, as for the DCs. To save bits, RUNLENGTH and SIZE are allocated only 4 bits each and squeezed into a single byte — let's call this *Symbol 1*. *Symbol 2* is the AMPLITUDE value; its number of bits is indicated by SIZE:

Symbol 1: (RUNLENGTH, SIZE)

Symbol 2: (AMPLITUDE)

JPEG Modes

The JPEG standard supports numerous modes (variations). Some of the commonly used ones are:

- Sequential Mode
- Progressive Mode
- Hierarchical Mode
- Lossless Mode

1. Sequential Mode

- This is the default JPEG mode. Each graylevel image or color image component is encoded in a single left-to-right, top-to-bottom scan.
- The “Motion JPEG” video codec uses Baseline Sequential JPEG, applied to each image frame in the video.

2. Progressive Mode

- Progressive JPEG delivers low quality versions of the image quickly, followed by higher quality passes, and has become widely supported in web browsers.
- Such multiple scans of images are of course most useful when the speed of the communication line is low.
- In Progressive Mode, the first few scans carry only a few bits and deliver a rough picture of what is to follow.
- After each additional scan, more data is received, and image quality is gradually enhanced.
- The advantage is that the user end has a choice whether to continue receiving image data after the first scan(s).

3. Hierarchical Mode

- Hierarchical JPEG encodes the image in a hierarchy of several different resolutions.
- The encoded image at the lowest resolution is basically a compressed low-pass-filtered image, whereas the images at successively higher resolutions provide additional details (differences from the lower-resolution images).

Similar to Progressive JPEG, Hierarchical JPEG images can be transmitted in multiple passes with progressively improving quality

4. Lossless Mode

- Lossless JPEG is a very special case of JPEG which indeed has no loss in its image quality.
- It employs only a simple differential coding method, involving no transform coding.
- It is rarely used, since its compression ratio is very low compared to other, lossy modes.

The JPEG 2000 Standard

- It is most successful and popular image format.
- The main reason for its success is the quality of its output for relatively good compression ratio.
- The new JPEG2000 standard aims to provide not only a better rate-distortion tradeoff and improved subjective image quality.

JPEG (Joint Photographic Experts Group) compression is a widely used method of lossy image compression, designed to reduce the file size of digital images while maintaining acceptable visual quality

- The main compression method used in JPEG2000 is the (Embedded Block Coding with Optimized Truncation) algorithm (EBCOT), designed by Taubman.
- EBCOT produces a bitstream with a number of desirable features, including quality and resolution scalability and random access.
- The basic idea of EBCOT is the partition of each sub-band LL, LH, HL, HH produced by the wavelet transform into small blocks called code blocks.
- Each code block is coded independently, in such a way that no information for any other block is used.
- The EBCOT algorithm consists of three steps:
 1. Block coding and bitstream generation
 2. Postcompression rate-distortion (PCRD) optimization
 3. Layer formation and representation.

1. Block coding and bitstream generation

- Each sub-band generated for the 2D discrete wavelet transform is first partitioned into small code blocks, typically 64×64 , or other size no less than 32×32 .
- Then the EBCOT algorithm generates a highly scalable bitstream for each code block B_i .
- The bitstream associated with B_i may be independently truncated to any member of a predetermined collection of
 - different lengths R_i^n , with associated distortion D_i^n
- For each code block $s_i[k] = s_i[k_1, k_2]$ be the 2D sequence of small code blocks of sub-band samples, with k_1 and k_2 the row and column index.

2. Post Compression Rate-Distortion Optimization

- The goal of PCRD is to produce an optimal truncation of each code block's independent bitstream such that distortion is minimized, subject to the bit-rate constraint.
- For each truncated embedded bitstream of code block B_i having rate $R_i^{n_i}$

$$D = \sum_i D_i^{n_i}$$

the overall distortion of the reconstructed image is

Is the distortion from code block having truncation point.

3. Layer Formation and Representation

The compressed bit-stream is divided into **quality layers** to support different resolutions and quality levels.

Steps in Layer Formation:

1.Bit-plane Coding:

- The image is transformed using Discrete Wavelet Transform (DWT).
- Each sub-band is split into small code blocks (e.g., 64×64 or 32×32 pixels).
- These blocks are encoded bit-plane by bit-plane from most significant bit (MSB) to least significant bit (LSB).

2.Pass Partitioning:

- Each bit-plane is divided into three coding passes:

- Significance Propagation Pass (Identifies new significant pixels)
- Magnitude Refinement Pass (Refines existing significant pixels)
- Cleanup Pass (Handles less important coefficients)

3 Layer Assignment:

- Different coding passes are grouped into quality layers based on rate-distortion optimization.
- Higher layers contain more details and better quality.

4. Truncation Points & Bit-Stream Formation:

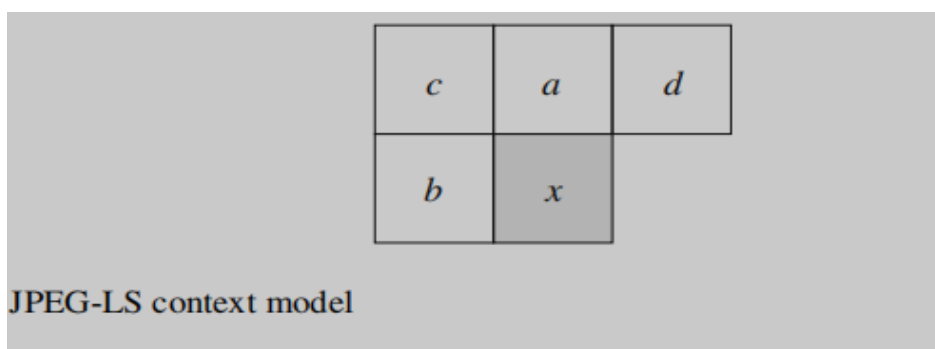
- The algorithm selects optimal truncation points for each block based on bit-rate constraints.
- The compressed data is stored in layers, allowing for progressive decoding.

The JPEG-LS Standard

- The main advantage of JPEG-LS over JPEG2000 is that JPEG-LS is based on a low complexity algorithm.
- JPEG-LS is part of a larger ISO effort aimed at better compression of medical images.
- JPEG-LS is in fact the current ISO/ITU standard for lossless or “near lossless” compression of continuous-tone images.
- The core algorithm in JPEG-LS is called LOw COmplexity LOssless COmpression for Images (LOCO-I), proposed by Hewlett-Packard.

The design of this algorithm is motivated by the observation that complexity reduction is often more important overall than any small increase in compression offered by more complex algorithms

- LOCO-I exploits a concept called context modeling.
- The idea of context modeling is to take advantage of the structure in the input source—conditional probabilities of what pixel values follow from each other in the image.
- This extra knowledge is called the context.
- If the input source contains substantial structure, as is usually the case, we could potentially compress it using fewer bits than the 0th- order entropy.



LOCO-I can be broken down into three components:

- **Prediction.** Predicting the value of the next sample x' using a causal template
- **Context determination.** Determining the context in which x' occurs
- **Residual coding.** Entropy coding of the prediction residual conditioned by the context of x'

➤ PREDICTION

- The fixed predictor used by the algorithm is given as follows:

$$\hat{x}' = \begin{cases} \min(a, b) & c \geq \max(a, b) \\ \max(a, b) & c \leq \min(a, b) \\ a + b - c & \text{otherwise} \end{cases}$$



Context Determination

The context model that conditions the current prediction error (the *residual*) is indexed using a three-component context vector $\mathbf{Q} = (q_1, q_2, q_3)$, whose components are

$$\begin{aligned} q_1 &= d - b \\ q_2 &= b - c \\ q_3 &= c - a \end{aligned} \tag{9.10}$$

These differences represent the local gradient that captures the local smoothness or edge contents that surround the current sample.

☐ Lower Q-values → More detail retention, lower compression efficiency.

☐ Higher Q-values → Better compression, possible detail loss.

➤ Residual Coding

Residual coding is a technique used in image and video compression to encode the difference (residual) between predicted and actual pixel values instead of encoding the pixel values directly.

Instead of storing the original data, predict a value and only encode the difference (residual). Since the residual is usually small, it requires fewer bits to encode, leading to better compression.



Near-Lossless Mode

The JPEG-LS standard also offers a near-lossless mode, in which the reconstructed samples deviate from the original by no more than an amount δ . The main lossless JPEG-LS mode can be considered a special case of the near-lossless mode with $\delta = 0$. Near-lossless com-

BILEVEL IMAGE COMPRESSION STANDARDS

As more and more documents are handled in electronic form, efficient methods for compressing bilevel images (those with only 1-bit, black-and-white pixels) are much in demand. A familiar example is fax images. Algorithms that take advantage of the binary nature of the image data often perform better than generic image-compression algorithms. Earlier

The JBIG Standard

JBIG is the coding standard recommended by the Joint Bi-level Image Processing Group for binary images. This lossless compression standard is used primarily to code scanned images of printed or handwritten text, computer-generated text, and facsimile transmissions. It offers progressive encoding and decoding capability, in the sense that the resulting bitstream contains a set of progressively higher-resolution images. This standard can also be used to code grayscale and color images by coding each bitplane independently, but this is not the main objective.

Modes of JBIG Compression

According to the provided image, JBIG has **three modes of operation**:

1. Progressive Mode

- The image is transmitted in multiple passes, gradually refining the quality.
- Useful for low-bandwidth applications where a rough preview is needed before full decoding.
- Each pass provides more details, improving resolution over time.

2. Progressive-Compatible Sequential Mode

- Uses a bitstream compatible with Progressive Mode, but the data is divided into strips.
- Allows easier error recovery and better processing in systems that require structured data.
- Still benefits from progressive-style compression but works in a sequential way.

3. Single-Progression Sequential Mode

- The image is transmitted in a single pass without multiple refinement steps.
- Faster compression and decoding, but no progressive refinement like the other two modes.
- Ideal for simple images and fast processing applications (e.g., scanning and fax transmission).

The JBIG encoder can be decomposed into two components:

- Resolution-reduction and differential-layer encoder
- Lowest-resolution-layer encoder

The input image goes through a sequence of resolution-reduction and differential-layer encoders. Each is equivalent in functionality, except that their input images have different resolutions. Some implementations of the JBIG standard may choose to recursively use one such physical encoder. The lowest-resolution image is coded using the lowest-resolution-layer encoder. The design of this encoder is somewhat simpler than that of the resolution-reduction and differential-layer encoders, since the resolution-reduction and deterministic-prediction operations are not needed.

The JBIG2 Standard

- The design goal for JBIG2 aims not only at providing superior lossless compression performance over existing standards but also at incorporating lossy compression at a much higher compression ratio, with as little visible degradation as possible.
- A unique feature of JBIG2 is that it is both quality progressive and content progressive.

The JBIG2 encoder decomposes the input bi-level image into regions of different attributes and codes each separately, using different coding methods

Model-Based Coding

- The idea behind model-based coding is essentially the same as that of context-based coding.
- If we can separate the image content into different categories and derive a model specifically for each, we are much more likely to accurately model the behavior of the data and thus achieve higher compression ratio.
- In the JBIG style of coding, adaptive and model templates capture the structure within the image.
- This model is general, in the sense that it applies to all kinds of data.

Text Region Coding

- Each text region is further segmented into pixel blocks containing connected black pixels.
- These blocks correspond to characters that make up the content of this region.
- Then, instead of coding all pixels of each character, the bitmap of one representative instance of this character is coded and placed into a dictionary.
- For any character to be coded, the algorithm first tries to find a match with the characters in the dictionary.
- If one is found, then both a pointer to the corresponding entry in the dictionary and the position of the character on the page are coded.
- The pixel block is coded directly and added to the dictionary.

Preprocessing and Postprocessing

- From the decoder point of view, the decoded bitstream is lossless with respect to the image encoded by the encoder, although not necessarily with respect to the original image.
- The encoder may modify the input image in a preprocessing step, to increase coding efficiency
 - The preprocessor usually tries to change the original image to lower the code length in a way that does not generally affect the image's appearance.
 - It tries to remove noisy pixels and smooth out pixel blocks.